

数据库系统 Ch9 —数据库设计

详细学习笔记

基于：数据库系统 PPT Ch9 DataBase Design (55 页)

2026 年 6 月 21 日

目录

1 数据库设计概述 [p.36]	2
2 数据库物理设计概述 [p.1–3]	2
2.1 定义 [p.1]	2
2.2 物理设计步骤 [p.2–3]	2
3 物理设计的内容和方法 (9.5.1) [p.4–8]	2
3.1 准备工作 [p.5]	2
3.2 选择物理设计所需参数 [p.6–7]	2
3.3 物理设计的内容 [p.7]	3
4 关系模式存取方法选择 (9.5.2) [p.9–23]	3
4.1 概述 [p.9–10]	3
4.2 一、索引存取方法选择 [p.11–12]	3
4.3 二、聚簇存取方法选择 [p.13–22]	3
4.3.1 聚簇定义 [p.13–14]	3
4.3.2 聚簇的优点 [p.16–17]	3
4.3.3 聚簇的局限性和适用范围 [p.15, 18–20]	4
4.3.4 组合聚簇 [p.19]	4
4.3.5 选择聚簇的设计步骤 [p.21–22]	4
4.4 三、HASH 存取方法选择 [p.23]	4
5 确定数据库的存储结构 (9.5.3) [p.24–32]	4
5.1 确定数据的存放位置 [p.25–29]	4

5.1.1 实例 [p.28–29]	4
5.2 确定系统配置 [p.30–31]	5
5.3 MySQL 物理存储补充 [p.32]	5
6 评价物理结构 (9.5.4) [p.33–35]	5
6.1 评价内容 [p.34]	5
6.2 评价方法 [p.35]	5
7 数据库实施 (9.6) [p.37–41]	5
7.1 实施内容 [p.37–38]	5
7.2 定义数据库结构 [p.39–41]	6
8 JDBC 补充 [p.42–54]	6
8.1 JDBC 简介 [p.43]	6
8.2 JDBC 基本步骤 [p.45]	6
8.3 关键技术点 [p.46–50]	6
8.4 完整 JDBC 示例 [p.53–54]	6

1 数据库设计概述 [p.36]

- **数据库设计 (Database Design)**: 为给定应用环境构造最优数据库模式、建立数据库及其应用系统的过程
- 本章涉及阶段: 需求分析 (9.2) → 概念结构设计 (9.3) → 逻辑结构设计 (9.4) → **物理设计 (9.5)** → **数据库实施 (9.6)** → 运行与维护 (9.7)

2 数据库物理设计概述 [p.1–3]

2.1 定义 [p.1]

- **数据库物理设计 (Physical Database Design)**: 为给定的逻辑数据模型选取最适合应用环境的物理结构的过程
- **物理结构**: 数据库在物理设备上的**存储结构与存取方法**, 依赖于给定的计算机系统

2.2 物理设计步骤 [p.2–3]

1. **确定数据库的物理结构** (存储结构 + 存取方法)
2. **评价物理结构**: 重点评价时间和空间效率
3. 若满足设计要求 → 物理实施; 否则 → 重新设计或修改物理结构, 甚至返回逻辑设计阶段修改数据模型

3 物理设计的内容和方法 (9.5.1) [p.4–8]

3.1 准备工作 [p.5]

- 充分了解**应用环境**, 详细分析要运行的事务, 获得选择物理设计所需参数
- 充分了解所用 **RDBMS** 的内部特征, 特别是系统提供的存取方法和存储结构

3.2 选择物理设计所需参数 [p.6–7]

- **数据库查询事务**需要分析 [p.6]:
 - 查询的关系、查询条件所涉及的属性
 - 连接条件所涉及的属性、查询的投影属性
- **数据更新事务**需要分析 [p.7]:
 - 被更新的关系、更新操作条件涉及的属性
 - 修改操作要改变的属性值
 - 每个事务在各关系上运行的频率和性能要求

3.3 物理设计的内容 [p.7]

1. 为关系模式选择存取方法（建立存取路径）
2. 设计关系、索引等数据库文件的物理存储结构

4 关系模式存取方法选择 (9.5.2) [p.9–23]

4.1 概述 [p.9–10]

- 数据库系统是多用户共享系统，对同一关系需建立多条存取路径满足多种应用
- DBMS 常用存取方法：索引方法（主要是 B+ 树）、聚簇 (Cluster) 方法、HASH 方法 [p.10]

4.2 一、索引存取方法选择 [p.11–12]

- 选择内容：确定对哪些属性列建立索引/组合索引；哪些索引设计为唯一索引 [p.11]
- 选择规则 [p.12]:
 1. 频繁出现在查询条件中的属性 → 建立索引（或组合索引）
 2. 经常作为聚集函数参数（MAX/MIN 等）的属性 → 建立索引
 3. 经常在连接条件中出现的属性 → 建立索引
- 注意：索引数过多带来额外开销（维护开销 + 查找开销）
- 易错点：不是索引越多越好——每次增删改都需要同步维护索引

4.3 二、聚簇存取方法选择 [p.13–22]

4.3.1 聚簇定义 [p.13–14]

- 聚簇 (Cluster)：将聚簇码 (Cluster Key) 上具有相同值的元组集中存放在连续的物理块中
- 聚簇索引：建立聚簇索引后，基表中数据按指定聚簇属性值的升序/降序存放，索引项顺序与元组物理顺序一致 [p.14]
- 一个基本表上最多只能建立一个聚簇索引 [p.15]
- SQL 示例：CREATE CLUSTER INDEX Stusname ON Student(Sname); [p.14]

4.3.2 聚簇的优点 [p.16–17]

- 大大提高按聚簇属性查询的效率
- 例：信息系 500 名学生分布在 500 个不同物理块 → 至少 500 次 I/O；聚簇后每读一个物理块可得到多个满足条件的元组 [p.16]
- 节省存储空间：聚簇码值不必在每个元组中重复存储 [p.17]

4.3.3 聚簇的局限性和适用范围 [p.15, 18–20]

- **局限性：**只能提高某些特定应用性能；建立与维护开销大（元组物理迁移、原有索引重建） [p.18]
- 聚簇码值改变时 → 元组存储位置需相应移动
- **适用范围：**
 - 很少对基表进行增删操作 [p.15]
 - 很少对其中的变长列进行修改操作
 - 通过聚簇码访问或连接是**主要应用** [p.20]
 - SQL 中含 ORDER BY/GROUP BY/UNION/DISTINCT 时特别有利（省去排序） [p.20]

4.3.4 组合聚簇 [p.19]

- 多个关系按“预连接”形式聚簇存放
- 例：将具有相同学号的学生元组和选课元组在物理上聚簇在一起 → 提高连接效率 [p.19]

4.3.5 选择聚簇的设计步骤 [p.21–22]

1. **设计候选聚簇：**对经常连接操作的关系建立组合聚簇；频繁相等比较的属性建立聚簇；重复率高的属性建立聚簇 [p.21]
2. **检查并取消不必要的：**删除经常全表扫描的关系；删除更新远多于查询的关系；删除重复关系 [p.22]
3. **选择最优方案：**选总代价最小的聚簇方案

4.4 三、HASH 存取方法选择 [p.23]

- **选择规则：**当关系满足以下两个条件时可选择 HASH 存取方法
 1. 该关系的属性主要出现在**等值连接条件或相等比较**选择条件中
 2. 该关系的大小**可预知且不变**；或大小动态改变但 DBMS 提供动态 HASH 存取方法

5 确定数据库的存储结构 (9.5.3) [p.24–32]

5.1 确定数据的存放位置 [p.25–29]

- **内容：**确定关系/索引/聚簇/日志/备份的存放位置 [p.25]
- **权衡因素** [p.26]：存取时间、存储空间利用率、维护代价（三者常相互矛盾）
- **基本原则** [p.27]：易变部分与稳定部分分开存放；存取频率较高与较低部分分开存放

5.1.1 实例 [p.28–29]

- 备份/日志备份 → 考虑放磁带（只在故障恢复时使用，数据量大）

- 表和索引分别放不同磁盘 → 两个磁盘并行工作，加快读写
- 大表分别放两个磁盘 → 多用户环境特别有效
- 日志文件与数据库对象分开放 → 改进系统性能

5.2 确定系统配置 [p.30–31]

- DBMS 提供的**存储分配参数**：并发用户数/打开数据库对象数/缓冲区长度和个数/时间片大小/数据库大小/锁数目 [p.30]
- 系统提供合理默认值，但物理设计时需根据应用环境调整
- 物理设计的配置调整是**初步的**，运行时还需根据实际情况进一步调整 [p.31]

5.3 MySQL 物理存储补充 [p.32]

- **InnoDB 存储引擎文件类型**（最常用，支持事务 ACID 和行级锁）：
 - 表空间文件：ibdata1(系统表空间) / .ibd(独立表空间 per-table)
 - 日志文件：ib_logfile0/ib_logfile1 (Redo 日志，用于事务恢复)
 - 撤销日志：同上文件存储 Undo 日志（事务回滚时恢复数据）
 - 缓冲池：ib_buffer_pool（内存中缓存频繁访问的数据和索引）

6 评价物理结构 (9.5.4) [p.33–35]

6.1 评价内容 [p.34]

对物理设计过程中产生的多种方案进行细致评价，从中选择**较优方案**

6.2 评价方法 [p.35]

- **定量估算**各种方案的存储空间、存取时间、维护代价
- 对估算结果权衡比较，选择较优的合理物理结构
- 若不符合用户需求 → 修改设计

7 数据库实施 (9.6) [p.37–41]

7.1 实施内容 [p.37–38]

1. 用 **DDL** 定义数据库结构：用 DBMS 提供的 DDL 严格描述数据库结构 [p.39]
2. 组织数据入库：数据装载
3. 编制与调试应用程序
4. 数据库试运行

7.2 定义数据库结构 [p.39–41]

- 用 SQL 语句定义表结构: CREATE TABLE
- 定义视图: CREATE VIEW
- 如需要聚簇, 在建基本表前先用 CREATE CLUSTER 定义 [p.41]

8 JDBC 补充 [p.42–54]

8.1 JDBC 简介 [p.43]

- **JDBC** (Java DataBase Connectivity): Sun 公司提供的 Java 连接数据库技术, 由接口和类构成的 API
- 包含两个包: 核心 API(java.sql) + 扩展 API(javax.sql)

8.2 JDBC 基本步骤 [p.45]

1. 注册驱动: `Class.forName("com.mysql.jdbc.Driver");` [p.46]
2. 建立连接 (Connection): `DriverManager.getConnection(url, user, password);` [p.47]
3. 创建语句 (Statement): `conn.createStatement();` [p.48]
4. 执行 SQL: `st.executeQuery(sql);` 或 `st.executeUpdate();` [p.48–49]
5. 处理结果 (ResultSet): `rs.next() → rs.getString()/getInt()` [p.49]
6. 释放资源: 依次释放 `ResultSet → Statement → Connection` [p.50]

8.3 关键技术点 [p.46–50]

- 推荐 `Class.forName()` 注册驱动, 不依赖具体驱动类 [p.46]
- URL 格式: JDBC: 子协议: 子名称//主机名: 端口/数据库名? 属性名 = 属性值 [p.47]
- `PreparedStatement` 支持参数化查询, 防止 SQL 注入 [p.48]
- 资源释放原则: Connection 尽量晚创建、尽量早释放 (数据库连接是稀有资源, 不及时关闭会导致系统宕机) [p.50]

8.4 完整 JDBC 示例 [p.53–54]

```
Class.forName("com.mysql.jdbc.Driver");
Connection conn = DriverManager.getConnection(URL,USER,PWD);
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery("select * from user");
while(rs.next()){
    System.out.println(rs.getObject(1)+"\t"+rs.getObject(2));
}
// finally: rs.close() -> st.close() -> conn.close()
```